

01. Selection Sort :- select minimum & swap

13	46	24	52	20	9
----	----	----	----	----	---

^{min}
⑨ 46 24 52 20 13

Step-1

^{min}
13 24 52 20 46

Step-2

^{min}
20 52 24 46

Step-3

^{min}
24 52 46

Step-4

^{min}
46 52

Step-5

9, 13, 20, 24, 46, 52

Best		$O(n^2)$	$\Rightarrow$ Time Complexity
Worst		$O(1)$	$\Rightarrow$ Space Complexity
Average			

Code:-

```

public static void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min = i;
        for (int j = i+1; j < n; j++) {
            if (arr[j] < arr[min]) {
                min = j;
            }
        }
        // swap
        int temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
    }
}

```

min

13	46	24	52	20	9
	j	j	j	j	j

46 > 13

24 > 13

52 > 13

20 > 13

9 < 13

min $\Rightarrow$ 5

9 46 24 52 20 13

02. Bubble Sort :- push the max to the last by adjacent swaps.

|13|46|24|52|20|9|

13, 46, 24, 52, 20, 9

-: Step 01

13, 24, 46, 52, 20, 9

13, 24, 46, 52, 20, 9

13, 24, 46, 20, 52, 9

13, 24, 46, 20, 9, 52 $\Rightarrow$ max

13, 24, 46, 20, 9, 52

-: Step 2

13, 24, 46, 20, 9, 52

13, 24, 20, 46, 9, 52

13, 24, 20, 9, 46, 52

13, 24, 20, 9, 46, 52

-: Step 3

13, 20, 24, 9, 46, 52

13, 20, 9, 24, 46, 52

13, 24, 20, 9

-: Step 4

13, 9, 20, 24, 46, 52

13, 20, 9, 24, 46, 52

-: Step 5

13, 9, 20, 24, 46, 52

9, 13, 20, 24, 46, 52 -: Step 05

code:-

```
public int[] bubbleSort(int arr[]) {  
    int n = arr.length;  
    for (int i = n-1; i >= 0; i--) {  
        Boolean swapped = false; // Best case
```

```
        for (int j = 0; j < i; j++) {  
            if (arr[j] > arr[j+1]) {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
                swapped = true; // Best Case  
            }  
        }  
    }
```

```
    if (!swapped) { // Best Case  
        break; // When arr is sorted  
    }
```

```
    }
```

```
    return arr;  
}
```

Recursive

```
if (n == 1) return;  
int didSwap = 0;  
for (int j = 0; j <= n-2; j++) {  
    if (arr[j] > arr[j+1]) {  
        int temp = arr[j];  
        arr[j] = arr[j+1];  
        arr[j+1] = temp; didSwap = 1  
    }  
}
```

Time Com.	$\approx O(N^2) \rightarrow$ worst complexity, Avg
	$= O(N) \rightarrow$ Best
SC	$= O(1)$

03. Insertion Sort :- Takes an element and places it in its correct position.

Time Com. $\approx O(N^2)$ $\rightarrow$ worst, Average
 $= O(N)$ $\rightarrow$ Best

① 9 14 15 12 6 8 13
 $\hookrightarrow$ is 9 in the correct position? (yes)

② 9 14 15 12 6 8 13
 $\hookrightarrow$ Are they in the correct position whether only 14 yes

③ 9 14 15 12 6 8 13
 $\hookrightarrow$ is 15 in the correct position? yes

④ 9 14 15 12 6 8 13
 $\hookrightarrow$ is 12 in the correct position? no

9 14 12 15 6 8 13
 9 12 14 15 6 8 13

9 12 14 15 6 8 13
 $\hookrightarrow$ they are in correct position

⑤ 9 12 14 15 6 8 13
 $\hookrightarrow$ 9 12 14 6 15 8 13
 9 12 6 14 15 8 13

9 6 12 14 15 8 13

6 9 12 14 15 8 13

⑥ 6 9 12 14 15 8 13

6 9 12 14 8 15 13

6 9 12 8 14 15 13

6 9 8 12 14 15 13

6 8 9 12 14 15 13

6 8 9 12 14 15 13

correct

⑦ 6 8 9 12 14 15 13

6 8 9 12 14 13 15

6 8 9 12 13 14 15

6 8 9 12 13 14 15

correct

Ans

Ans

6 8 9 12 13 14 15

Code:-

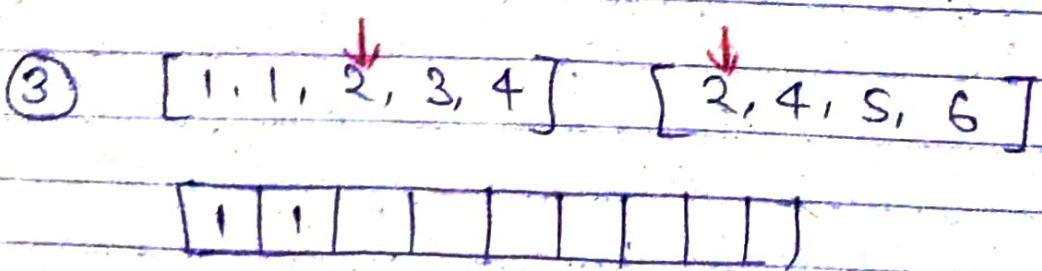
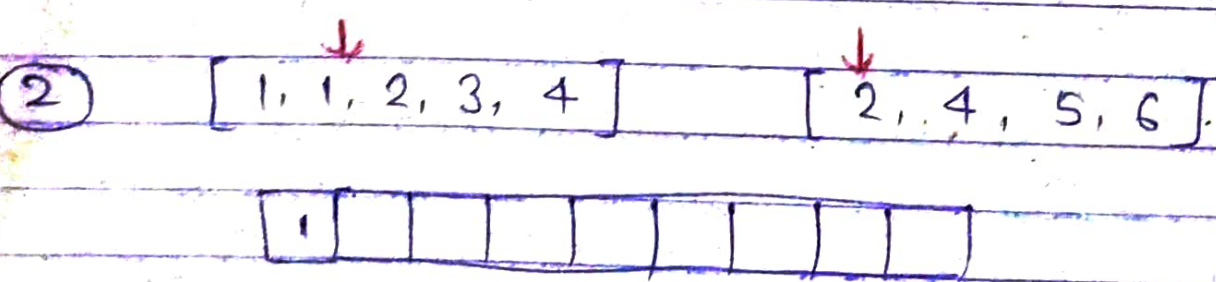
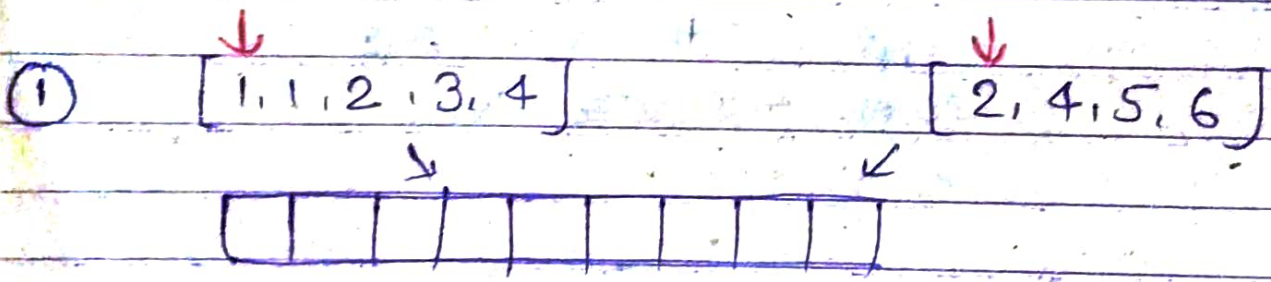
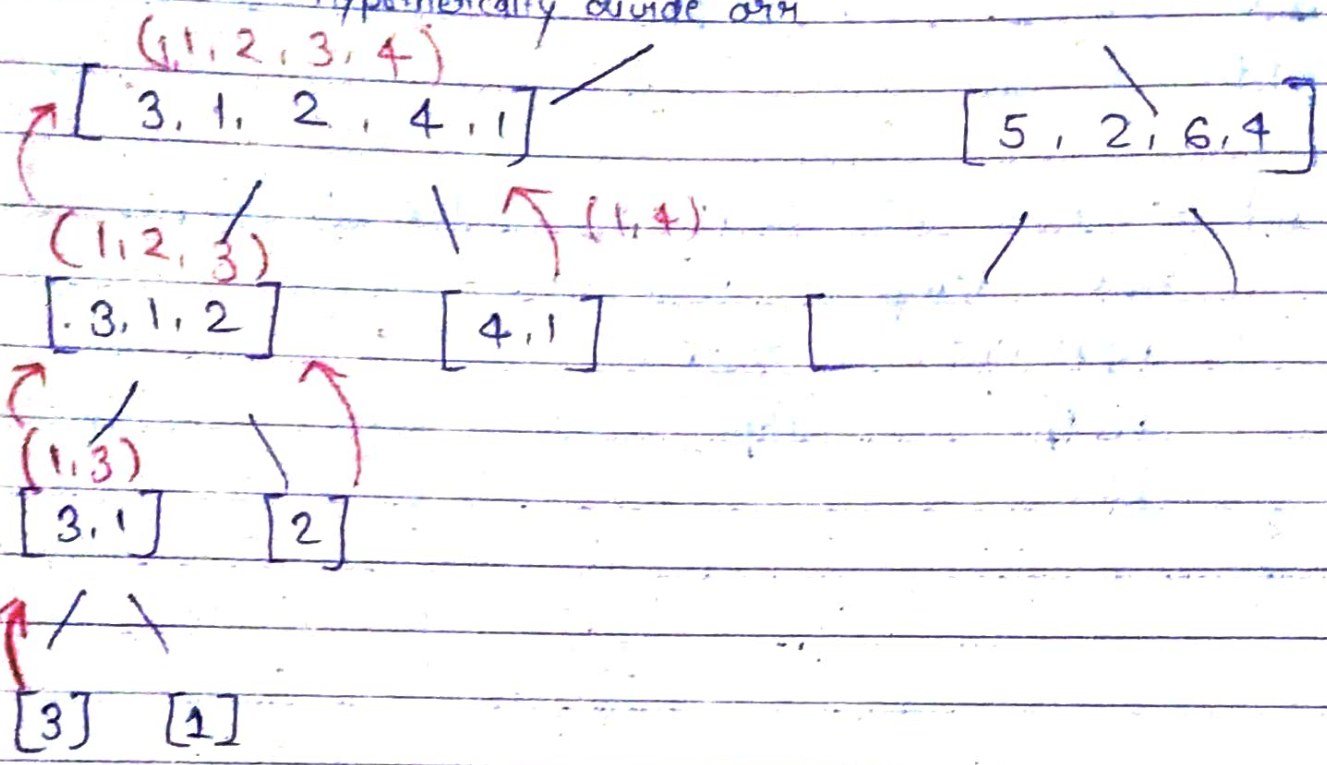
```
public static void insertionSort(int arr[], int n) {  
    for (int i = 0; i <= n-1; i++) {  
        int j = i;  
  
        while (j > 0 && arr[j-1] > arr[j]) {  
            int temp = arr[j-1];  
            arr[j-1] = arr[j];  
            arr[j] = temp;  
            j--;  
        }  
    }  
}
```

TC - $O(N^2)$ $\rightarrow$ Worst + Avg. $O(N)$ $\rightarrow$ Best Case
SC - $O(1)$

04. Merge Sort -

Array Run :- [3, 1, 2, 4, 1, 5, 2, 6, 4]

Hypothetically divide arr



④

[1, 1, 2, 3, 4]

[2, 4, 5, 6]

1	1	2							
---	---	---	--	--	--	--	--	--	--

⑤

[1, 1, 2, 3, 4]

[2, 4, 5, 6]

1	1	2	2	3					
---	---	---	---	---	--	--	--	--	--

⑥

[1, 1, 2, 3, 4]

[2, 4, 5, 6]

1	1	2	2	3	4				
---	---	---	---	---	---	--	--	--	--

⑦

[1, 1, 2, 3, 4]

[2, 4, 5, 6]

1	1	2	2	3	4	4	5	6	
---	---	---	---	---	---	---	---	---	--

Code: -

```
class Solution {  
    public int[] sortArray(int[] nums) {  
        mergeSort(nums, 0, nums.length-1);  
        return nums;  
    }  
}
```

```
    private void mergeSort(int arr[], int low,  
                           int high) {
```

```
        if (low >= high) return;  
        int mid = (low + high) / 2;  
        mergeSort(arr, low, mid);  
        mergeSort(arr, mid+1, high);  
        merge(arr, low, mid, high);  
    }
```

```
    private void merge(int arr[], int low,  
                       int mid, int high) {
```

```
        List<Integer> temp = new ArrayList<>();  
        int left = low;  
        int right = mid+1;
```

```
        while (left <= mid && right <= high) {
```

```
            if (arr[left] <= arr[right]) {  
                temp.add(arr[left++]);  
            } else {
```

```
                temp.add(arr[right++]);  
            }
```

```
        }
```

```

while (left <= mid) {
    temp.add (arr[left + 1]);
}

```

```

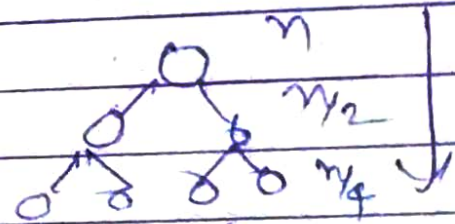
while (right <= high) {
    temp.add (arr[right + 1]);
}

```

```

for (int i = low; i < temp.length; i++) {
    arr[i] = temp.get(i - low);
}
arr[low + i] = temp[i];
}
}

```



TC - $O(N \log N)$

SC - $O(N) + O(\log N)$ Height of the tree $\rightarrow O(\log N)$
 $\hookrightarrow$ stack.

Out of Place

At left most subtree $\rightarrow$ left = 0 ✓

At right most subtree $\rightarrow$ it could be anything
 e.g. (5, 2, 3)

05. Quick Sort - (Divide & Conquer)

arr[] = [4, 6, 2, 5, 7, 9, 1, 3]

— Intuition —

(01) Pick a pivot & place it in its correct place in the sorted array.

(02) Smaller on the left and larger on the right.

Let's suppose we are picking pivot as first element in an array & place smaller to its left & greater to its right.

① [4, 6, 2, 5, 7, 9, 1, 3]

pivot = 4 (What's the correct place of 4?)

6 > 4 → right

2 < 4 → left

5 > 4 → right

7 > 4 → right

9 > 4 → right

1 < 4 → left

3 < 4 → left

[2, 1, 3, 4, 6, 5, 7, 9]

②

[2, 1, 3, 4, 6, 5, 7, 9]

pivot = 2 (What's the correct place of 2?)

1 < 2 → left

3 > 2 → right

[1, 2, 3, 4, 6, 5, 7, 9]

③

[1, 2, 3, 4, 6, 5, 7, 9]

pivot = 1 (What's the correct place of 1?)

2 > 1 → right ✓

④

[1, 2, 3, 4, 6, 5, 7, 9]

left part is sorted now move to
its right part

[6, 5, 7, 9]

pivot ⇒ 6 (What's the correct position?)

5 < 6 → left

7 > 6 → right

[5, 6, 7, 9]

⑤ [7, 9]

pivot = 7 [what's the correct place of 7?]

9 > 7 → right

[7, 9]

⑥ [9] is the single element so that is also done..

NOW HOW WOULD YOU IMPLEMENT THIS LOGIC IN CODE :-

①

4	6	2	5	7	9	1	3
---	---	---	---	---	---	---	---

low high

pivot = arr[low]

②

4	6	2	5	7	9	1	3
---	---	---	---	---	---	---	---

low high
i j

Now find first element which is greater than pivot using i pointer & smaller than pivot using j pointer

Once you find both of them swap

4	6	2	5	7	9	1	3
---	---	---	---	---	---	---	---

low i j high j

swap $\Rightarrow (6, 3)$

4	3	2	5	7	9	1	6
---	---	---	---	---	---	---	---

low i j high

swap (5, 1)

4	3	2	1	7	9	5	6
---	---	---	---	---	---	---	---

low j i high

~~swap (7, 9)~~

But i & j crossed $\Rightarrow$

so, Now swap (low, j)

1	3	2	4	7	9	5	6
---	---	---	---	---	---	---	---

low $\downarrow$ high
partition

Quicksort (low, partition - 1)

Quicksort (partition + 1, high)

// Codes -

```

class Solution {
public static void quickSort (int arr[], int low,
                               int high) {
    if (low < high) {
        int pivotIndex = partition (arr, low, high);
        quickSort (arr, low, pivotIndex - 1);
        quickSort (arr, pivotIndex + 1, high);
    }
}

```

```

private static int partition (int arr[],
                               int low, int high) {
    int pivot = arr[low];
    int i = low + 1;
    int j = high;

```

```

    while (i <= j) {
        while (i <= high &&
               arr[i] <= pivot) {
            i++;
        }

```

```

        while (arr[j] > pivot) {
            j--;
        }

```

```

        if (i < j) {
            swap (arr, i, j);
        }

```

```

    }
    swap (arr, low, j);
    return j;
}

```

PAGE NO.
 DATE: / /

```
private static void swap( int arr[], int i,
                          int j ) {
```

```
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

}

}

It is in-place algorithm — ...

↗ pivot → smallest, largest (all the time)
worst case TC — $O(N^2)$

TC — $O(N \log N)$
SC — $O(1)$

$O(\log N)$

→ auxiliary space
due to recursion